

A MINI PROJECT ON

Finance Management Model Bot

Submitted for the partial fulfillment of the requirement for the degree of

Bachelors of Business Administration in Artificial Intelligence with Honors

Submitted by –

Ashwin Chhawaniya – 25030422028

Arnav Singh – 25030422022

Akshat Goyal – 25030422014

(Semester – 1)

Submitted to –

DR. DAWA CHYOPHEL LEPCHA

Assistant Professor

Python Programming



SYMBIOSIS ARTIFICIAL INTELLIGENCE INSTITUTE

CONSTITUENT OF SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

LAVAL, MULSHI, PUNE-411211, MAHARASHTRA

November 2025

CERTIFICATE

This is to certify that the mini project titled **Finance Management Model Bot** has been carried out by **Ashwin Chhawaniya (25030422028)**, **Arnav Singh (25030422022)**, and **Akshat Goyal (25030422014)** of **Bachelor of Business Administration in Artificial Intelligence with Honors (Semester – 1)**, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Business Administration in Artificial Intelligence with Honors**.

The project has been completed under the supervision and guidance of **Dr. Dawa Chyophel Lepcha**, Assistant Professor – Python Programming, at the **Symbiosis Artificial Intelligence Institute**, a Constituent of **Symbiosis International (Deemed University)**, **Lavale, Mulshi, Pune – 411211, Maharashtra**, during the academic year **2025**.

Dr. Dawa Chyophel Lepcha

Assistant Professor, Python Programming

Head of the Department

Symbiosis Artificial Intelligence Institute

Constituent of **Symbiosis International (Deemed University)**

Lavale, Mulshi, Pune – 411211, Maharashtra

ACKNOWLEDGMENT

We would like to express our heartfelt gratitude to **Dr. Shruti Patil**, *Director, Symbiosis Artificial Intelligence Institute (SAII)*, for providing us with the opportunity, facilities, and academic environment necessary to successfully complete our mini project titled “**Finance Management Model Bot.**” Her leadership and encouragement have been a constant source of inspiration.

We are profoundly thankful to our project guide, **Dr. Dawa Chyophel Lepcha**, *Assistant Professor – Python Programming*, for his invaluable guidance, support, and insightful feedback throughout the course of this project. His mentorship has played a vital role in shaping our understanding of Python and its practical applications in business and finance.

We also extend our gratitude to all the faculty members of **SAII**, whose teaching and support have strengthened our knowledge foundation. Finally, we thank our parents, classmates, and friends for their unwavering encouragement, cooperation, and motivation, which greatly contributed to the successful completion of this project.

Ashwin Chhawaniya – 25030422028

Arnav Singh – 25030422022

Akshat Goyal – 25030422014

DECLARATION

We hereby declare that the mini project titled **FINANCE MANAGEMENT MODEL BOT** has been carried out by **ASHWIN CHHAWANIYA (25030422028)**, **ARNAV SINGH (25030422022)**, and **AKSHAT GOYAL (25030422014)** of **BACHELOR OF BUSINESS ADMINISTRATION IN ARTIFICIAL INTELLIGENCE WITH HONORS (SEMESTER – 1)**, under the guidance of **DR. DAWA CHYOPHEL LEPCHA, ASSISTANT PROFESSOR – PYTHON PROGRAMMING**, at the **SYMBIOSIS ARTIFICIAL INTELLIGENCE INSTITUTE (SAII)**, a constituent of **SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)**, Lavale, Mulshi, Pune – 411211, Maharashtra.

We further declare that this project report is the result of our original work and has not been submitted to any other university or institution for the award of any degree or diploma. All sources of information used in the preparation of this report have been duly acknowledged.

Place: SYMBIOSIS ARTIFICIAL INTELLIGENCE INSTITUTE (SAII), SIT BUILDING, 6TH FLOOR

Date: _____ NOVEMBER 2025

Ashwin Chhawaniya – 25030422028

Arnav Singh – 25030422022

Akshat Goyal – 25030422014

ABSTRACT

The mini project titled **FINANCE MANAGEMENT MODEL BOT** is developed using **PYTHON** as the core programming language and **MYSQL** as the database for data storage and management. The primary objective of this project is to design an intelligent, user-friendly financial assistant that helps users efficiently manage, analyze, and forecast their finances.

The system allows users to record income and expenses, plan monthly budgets, calculate taxes, compute compound interest, and perform real-time currency conversions. It also provides visual insights through graphical representations using **MATPLOTLIB**, thereby enhancing decision-making based on financial trends and savings growth.

The project demonstrates the effective integration of **PYTHON PROGRAMMING**, **DATABASE CONNECTIVITY**, and **DATA VISUALIZATION** techniques to create a functional, scalable, and interactive finance management tool. It serves as a practical implementation of theoretical knowledge and highlights the growing role of automation and artificial intelligence in the field of financial management.

Ashwin Chhawaniya – 25030422028

Arnav Singh – 25030422022

Akshat Goyal – 25030422014

FMM - Manual

College Finance Bot: A Personal Finance Management Tool

Project Report Version: 1.0 Date: November 2, 2025

Table of Contents

Section	Topic / Description
1.	Introduction <i>Background, motivation, objectives, and scope of the project.</i>
2.	Problem Statement <i>Define the problem you are solving and its relevance.</i>
3.	Literature Review / Related Work <i>Summary of existing studies or systems related to your topic.</i>
4.	Methodology Explain the approach, algorithms, software, or tools used. Include block diagrams, flowcharts, or pseudocode if applicable.
5.	Implementation / Experimental Setup Describe how you implemented your project (hardware/software environment). Include screenshots or results of program execution for a coding project.
6.	Results and Discussion Present findings using figures, graphs, and comparative tables. Discuss observations and performance outcomes.
7.	Applications <i>Describe real-world use or potential of your project.</i>
8.	Limitations <i>Mention what constraints or challenges you faced.</i>
9.	Conclusion and Future Work <i>Summarize what you achieved and suggest improvements or extensions.</i>
10.	References <i>Cite all sources used (books, papers, websites, etc.) in proper academic format.</i>
11.	Appendices

Section	Topic / Description
	<i>Include program code, circuit diagrams, or additional data not in the main text.</i>

1. Introduction

Background

Managing personal finances is a critical life skill, yet it is one that many college students and young professionals struggle with. The transition to financial independence often comes with new challenges, such as budgeting for tuition, rent, groceries, and social activities, while also considering long-term goals like saving and investing.

Motivation

The motivation for this project stems from the lack of simple, all-in-one, and free tools tailored for this demographic. Many existing financial applications are overly complex, designed for corporate use, or lock essential features behind a subscription. A simple, command-line-based tool can be both powerful and educational, encouraging users to actively engage with their finances.

Objectives

The primary objectives of this project are:

- To develop a multi-functional personal finance application.
- To provide core tools for tracking, budgeting, and forecasting.
- To ensure data persistence by using a robust database.
- To create a tool that is fast, responsive, and runs locally.
- To incorporate data visualization for complex topics like compound interest.

Scope

The scope of this project is a Python-based, command-line interface (CLI) application named "College Finance Bot." It interacts with a MySQL database to store user data. The application includes user authentication (registration and login) and a main menu providing access to seven core financial modules: a Monthly Budget Planner, Bill Splitter, Tax Calculator, Compound Interest Calculator, Currency Converter, Savings Calculator, and a primary Income/Expense Tracker.

2. Problem Statement

Students and young adults often lack a clear understanding of their financial health. They face a fragmented toolset: a separate app for budgeting, a calculator for splitting bills, a website for currency conversion, and complex spreadsheets for investment planning. This creates a high barrier to entry for effective financial management.

The central problem this project solves is the **lack of an accessible, consolidated, and educational financial tool for individuals with basic technical skills**. By integrating seven essential financial tools into a single, interactive CLI, the "College Finance Bot" provides a unified platform for users to manage their money, plan for the future, and learn about financial concepts in a hands-on manner.

3. Literature Review / Related Work

A review of the current market reveals several categories of financial tools:

- **Commercial GUI Applications (e.g., YNAB, Mint):** These are powerful, web-based tools with extensive features like bank account syncing and credit score monitoring. However, they are often subscription-based, and their complexity can be overwhelming for a beginner.
- **Plain-Text Accounting (e.g., `ledger-cli`, `hledger`):** These are powerful, command-line-based tools favored by technical users for their robustness and version-control capabilities. Their primary drawback is a steep learning curve; they are not interactive and require users to learn a specific syntax for making ledger entries.
- **Simple Mobile Apps (e.g., Splitwise):** These apps are highly effective at solving one specific problem (e.g., splitting bills) but do not offer a consolidated financial view.

This project, the "College Finance Bot," occupies a unique middle ground. It provides the simplicity and interactivity of a modern application but retains the speed and local-first feel of a command-line tool. It bridges the gap between single-use apps and overly complex accounting software.

4. Methodology

Approach

The project is built using a modular, function-driven approach in Python. Each financial tool is encapsulated within its own function (e.g., `bill_splitter()`, `compound_interest()`), which is called from a central control loop. This makes the code easy to maintain, debug, and extend.

Software and Tools

Tool / Library	Category	Purpose in Project
Python 3	Programming Language	The core language for all application logic.
MySQL	Database	Relational database for persistent storage of all user data.
<code>mysql-connector</code>	Python Library	The "bridge" or driver that allows Python to communicate with MySQL.
<code>matplotlib</code>	Python Library	Used for data visualization; generates graphs for financial forecasts.
<code>forex-python</code>	Python Library	Fetches live currency exchange rates from an online API.

System Architecture (Flowchart)

The flow of the application is as follows:

1. **Start:** The program initializes and calls `create_all_tables()` to ensure the database structure is ready.
2. **Authentication Gate (Loop):** The user is forced to either `register_user()` or `login_user()`. The loop only breaks upon a successful login, which returns a `user_id`.
3. **Main Menu (Loop):** The program enters the main operational loop.
 - a. `main_menu()` is displayed, and the user's choice is captured.
 - b. An `if/elif/else` block directs the program to the chosen function (e.g., `choice == "1"` calls `monthly_budget_planner(user_id)`).
 - c. The `user_id` is passed to the function, which is used in all SQL queries to ensure the user only sees and modifies their own data.
 - d. The function executes (gathers input, performs calculations, reads/writes to the database).
 - e. Control returns to the main menu loop.
4. **Logout:** If the user selects "8", the main menu loop is broken, and the program terminates.

5. Implementation / Experimental Setup

Software Environment

The project was developed and tested in a Python 3.8 environment. The database backend is a MySQL 8.0 server running on `localhost`. All required Python libraries (`mysql-connector-python`, `matplotlib`, `forex-python`) are installed via pip.

Database Implementation

The database (`college_finance_db`) is designed with eight normalized tables to prevent data redundancy:

Table Name	Purpose
<code>users</code>	Stores user login credentials (username, password).
<code>entries</code>	The main transaction ledger for all income and expenses.
<code>monthly_budget</code>	Stores budget plans created by the user.
<code>bill_splits</code>	A log of all calculations from the bill splitter.
<code>tax_calculations</code>	A log of all tax calculations.
<code>compound_interest_records</code>	Stores parameters and results of compound interest forecasts.
<code>currency_conversions</code>	A log of all currency conversions performed.
<code>savings_records</code>	Stores parameters and results of savings forecasts.

Key Feature Implementation: Compound Interest

The `compound_interest()` function provides a clear example of the project's methodology.

1. It collects user input for principal, contribution, years, rate, and variance.
2. It creates three scenarios: `rate - variance`, `rate`, and `rate + variance`.
3. It loops through each scenario, and for each, runs an inner loop to simulate the investment's growth month by month.
4. It uses `matplotlib.pyplot` to plot all three scenarios on a single line graph.
5. `plt.show()` is called, which pauses the program and displays the graph in a new window, providing an immediate visual result to the user.
6. The parameters and the final amount for the *target* rate are saved to the `compound_interest_records` table.

6. Results and Discussion

Program Execution

The application launches successfully from the command line. The user is first presented with the login/register prompt.

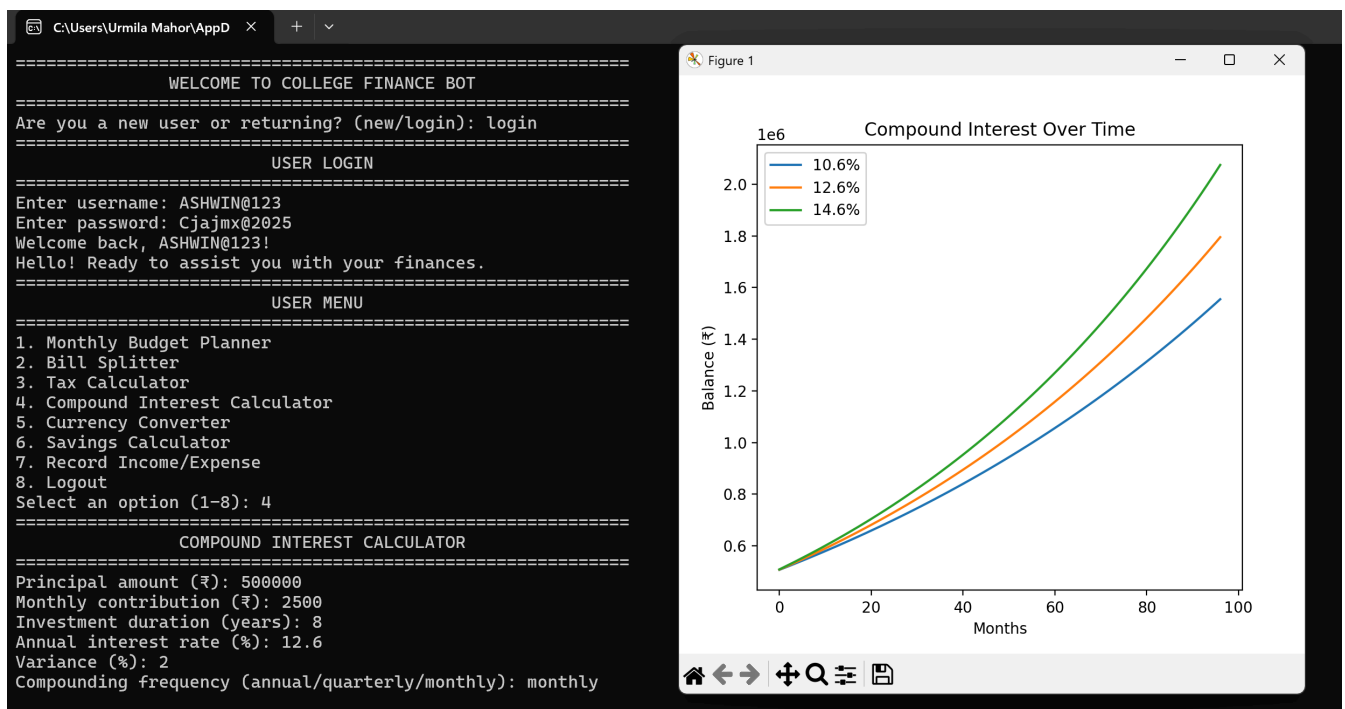
```
C:\Users\Urmila Mahor\AppData \ + v
=====
WELCOME TO COLLEGE FINANCE BOT
=====
Are you a new user or returning? (new/login): new
=====
NEW USER REGISTRATION
=====
Choose a username: ASHWIN@123
Choose a password: Cjajmx@2025
Confirm password: Cjajmx@2025
Registration successful! Please login now.
Are you a new user or returning? (new/login): login
```

Upon logging in, the main menu is displayed. All menu options are functional.

```
C:\Users\Urmila Mahor\AppData >
=====
WELCOME TO COLLEGE FINANCE BOT
=====
Are you a new user or returning? (new/login): login
=====
USER LOGIN
=====
Enter username: ASHWIN@123
Enter password: Cjajmx@2025
Welcome back, ASHWIN@123!
Hello! Ready to assist you with your finances.
=====
USER MENU
=====
1. Monthly Budget Planner
2. Bill Splitter
3. Tax Calculator
4. Compound Interest Calculator
5. Currency Converter
6. Savings Calculator
7. Record Income/Expense
8. Logout
Select an option (1-8): |
```

Results

- **Income/Expense Tracking:** The `record_entry` function correctly logs transactions to the `entries` table. The `get_user_financial_summary` function accurately computes the user's real-time balance, which is displayed after every transaction.
- **Forecasting Visualization:** The `compound_interest` and `savings_calculator` functions successfully generate and display graphs. The `matplotlib` window opens as expected and clearly shows the exponential growth, providing a powerful visual incentive for saving.



Discussion

The modular design proved highly effective. Adding new tools is straightforward: one only needs to create a new function and add an `elif` condition to the main loop. The use of a MySQL database for persistence is a significant advantage over simple file-based storage, as it provides security, scalability, and data integrity (e.g., using `ENUM` types for transaction categories).

The interactive nature of the CLI, combined with immediate visual feedback from `matplotlib`, makes the tool engaging. The user is not just entering data but immediately seeing the results and forecasts, which supports the project's educational objective.

7. Applications

- **Personal Finance Management:** The primary application is for students or any individual to manage their daily finances, track spending, and plan budgets from a simple terminal.
- **Educational Tool:** The application can be used in financial literacy workshops to demonstrate core concepts. The compound interest calculator, in particular, is a powerful tool for teaching the long-term benefits of saving.
- **Prototyping:** The project serves as a robust backend and logic prototype for a more complex, full-stack web application.

8. Limitations

While the project met its core objectives, it has several limitations:

- **Security:** This is the most significant limitation. Passwords are stored in plain text in the `users` table. This is a critical security flaw and would be unacceptable in a production environment.
- **User Interface:** The application is purely CLI-based. This makes it inaccessible to non-technical users who are accustomed to graphical user interfaces (GUIs).
- **Network Dependency:** The `forex-python` library requires an internet connection to fetch live rates. Although a fixed-rate fallback is implemented, the primary feature depends on connectivity.
- **Error Handling:** The error handling is basic (using `try...except Exception as e`). A more robust implementation would catch specific errors (e.g., `mysql.connector.Error`) and provide more targeted feedback to the user.

9. Conclusion and Future Work

Conclusion

This project successfully demonstrates the creation of a functional, consolidated, and interactive personal finance tool. The "College Finance Bot" meets its objective of providing a simple, powerful, and educational platform for managing budgets, tracking expenses, and forecasting financial goals. The integration of a persistent database and data-visualization tools proves to be a highly effective combination, even within a simple CLI.

Future Work

There are several clear paths for extending this project:

1. **Implement Password Hashing:** The top priority is to fix the security flaw by using a library like `bcrypt` to hash and salt all user passwords before storing them.
2. **Develop a GUI:** The modular backend logic can be decoupled from the CLI and used to power a web application (using a framework like `Flask` or `React`) or a desktop GUI (using `Tkinter` or `PyQt`).
3. **Add Data Analysis and Reporting:** Create a new module to analyze historical spending data from the `entries` table, showing the user their spending habits by tag or category, and allowing them to export reports (e.g., to a

CSV file).

4. **Bank Account Integration:** For a production-level tool, explore using APIs like Plaid to securely sync bank accounts and automate transaction entries.

10. References

- Python 3 Official Documentation. (2025). *The Python Standard Library*. Retrieved from docs.python.org
- MySQL Documentation. (2025). *MySQL 8.0 Reference Manual*. Retrieved from <https://www.google.com/search?q=dev.mysql.com/doc>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95.
- mysql-connector-python Developer Guide. (2025). Retrieved from <https://www.google.com/search?q=dev.mysql.com/doc/connector-python/en/>
- forex-python Library Documentation. (2025). Retrieved from pypi.org/project/forex-python/

11. Appendices

Appendix A: Full Program Code

```
import mysql.connector
import datetime
import matplotlib.pyplot as plt
from forex_python.converter import CurrencyRates

DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": "javamx",
    "database": "college_finance_db"
}

CURRENCIES = {
    "INR": "Indian Rupee",
    "USD": "US Dollar",
    "EUR": "Euro",
    "GBP": "British Pound",
    "JPY": "Japanese Yen",
    "AUD": "Australian Dollar",
    "CAD": "Canadian Dollar",
    "CHF": "Swiss Franc",
    "CNY": "Chinese Yuan",
    "NZD": "New Zealand Dollar",
}

conversion_rates = {
    "USD": {"INR_to_currency": 0.01133, "currency_to_INR": 88.28},
    "EUR": {"INR_to_currency": 0.00965, "currency_to_INR": 103.60},
    "GBP": {"INR_to_currency": 0.00835, "currency_to_INR": 119.66},
    "JPY": {"INR_to_currency": 1.67, "currency_to_INR": 0.598},
    "AUD": {"INR_to_currency": 0.01722, "currency_to_INR": 58.08},
    "CAD": {"INR_to_currency": 0.0141, "currency_to_INR": 70.92},
    "CHF": {"INR_to_currency": 0.0108, "currency_to_INR": 92.60},
    "CNY": {"INR_to_currency": 0.0807, "currency_to_INR": 12.29},
    "NZD": {"INR_to_currency": 0.0190, "currency_to_INR": 52.59},
```

```

}

def convert_inr_to_currency(amount_inr, currency_code):
    if currency_code not in conversion_rates:
        raise ValueError("Unsupported currency code")
    rate = conversion_rates[currency_code]["INR_to_currency"]
    return amount_inr * rate

def convert_currency_to_inr(amount_currency, currency_code):
    if currency_code not in conversion_rates:
        raise ValueError("Unsupported currency code")
    rate = conversion_rates[currency_code]["currency_to_INR"]
    return amount_currency * rate

def print_banner(text):
    print("=" * 60)
    print(text.center(60))
    print("=" * 60)

def db_connect():
    try:
        cnx = mysql.connector.connect(**DB_CONFIG)
        cursor = cnx.cursor()
        return cnx, cursor
    except mysql.connector.Error as err:
        print(f"Failed to connect to database: {err}")
        exit(1)

def create_all_tables():
    cnx = mysql.connector.connect(**{k: v for k, v in DB_CONFIG.items() if k !=
"database"})
    cursor = cnx.cursor()
    try:
        cursor.execute(f"CREATE DATABASE IF NOT EXISTS {DB_CONFIG['database']} DEFAULT
CHARACTER SET utf8")
    except mysql.connector.Error as err:
        print(f"Failed creating database: {err}")
        exit(1)
    cnx.database = DB_CONFIG["database"]
    table_ddls = {
        "users": """
            CREATE TABLE IF NOT EXISTS users (
                id INT AUTO_INCREMENT PRIMARY KEY,
                username VARCHAR(50) UNIQUE NOT NULL,
                password_hash VARCHAR(255) NOT NULL
            ) ENGINE=InnoDB
        """,
        "monthly_budget": """
            CREATE TABLE IF NOT EXISTS monthly_budget (
                id INT AUTO_INCREMENT PRIMARY KEY,

```

```

        user_id INT NOT NULL,
        month VARCHAR(7) NOT NULL,
        budget FLOAT NOT NULL,
        priority1 VARCHAR(100),
        priority1_percent INT,
        priority2 VARCHAR(100),
        priority2_percent INT,
        priority3 VARCHAR(100),
        priority3_percent INT,
        miscellaneous_percent INT
    ) ENGINE=InnoDB
""",
"bill_splits": ""
    CREATE TABLE IF NOT EXISTS bill_splits (
        id INT AUTO_INCREMENT PRIMARY KEY,
        user_id INT NOT NULL,
        bill_amount FLOAT NOT NULL,
        num_people INT NOT NULL,
        per_person_share FLOAT NOT NULL,
        timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
    ) ENGINE=InnoDB
""",
"tax_calculations": ""
    CREATE TABLE IF NOT EXISTS tax_calculations (
        id INT AUTO_INCREMENT PRIMARY KEY,
        user_id INT NOT NULL,
        income FLOAT NOT NULL,
        tax FLOAT NOT NULL,
        effective_rate FLOAT NOT NULL,
        timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
    ) ENGINE=InnoDB
""",
"compound_interest_records": ""
    CREATE TABLE IF NOT EXISTS compound_interest_records (
        id INT AUTO_INCREMENT PRIMARY KEY,
        user_id INT NOT NULL,
        principal FLOAT NOT NULL,
        monthly_contrib FLOAT NOT NULL,
        years FLOAT NOT NULL,
        annual_rate FLOAT NOT NULL,
        variance FLOAT NOT NULL,
        frequency ENUM('annual', 'quarterly', 'monthly'),
        final_amount FLOAT,
        timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
    ) ENGINE=InnoDB
""",
"currency_conversions": ""
    CREATE TABLE IF NOT EXISTS currency_conversions (
        id INT AUTO_INCREMENT PRIMARY KEY,
        user_id INT NOT NULL,
        from_currency VARCHAR(3) NOT NULL,
        to_currency VARCHAR(3) NOT NULL,
        amount FLOAT NOT NULL,
        converted_amount FLOAT NOT NULL,
        timestamp DATETIME DEFAULT CURRENT_TIMESTAMP

```

```

        ) ENGINE=InnoDB
    """
    "savings_records": """
        CREATE TABLE IF NOT EXISTS savings_records (
            id INT AUTO_INCREMENT PRIMARY KEY,
            user_id INT NOT NULL,
            monthly_saving FLOAT NOT NULL,
            years FLOAT NOT NULL,
            annual_return FLOAT NOT NULL,
            final_balance FLOAT,
            timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
        ) ENGINE=InnoDB
    """
    "entries": """
        CREATE TABLE IF NOT EXISTS entries (
            id INT AUTO_INCREMENT PRIMARY KEY,
            user_id INT NOT NULL,
            type ENUM('income','expense'),
            amount FLOAT NOT NULL,
            tag VARCHAR(100) NOT NULL,
            entry_time DATETIME NOT NULL
        ) ENGINE=InnoDB
    """
}
for name, ddl in table_ddls.items():
    try:
        cursor.execute(ddl)
    except mysql.connector.Error as err:
        print(f"Error creating table {name}: {err}")
        exit(1)
cursor.close()
cnx.close()

```

```

def register_user():
    cnx, cursor = db_connect()
    print_banner("NEW USER REGISTRATION")
    while True:
        username = input("Choose a username: ").strip()
        if len(username) < 3:
            print("Username must be at least 3 characters.")
            continue
        cursor.execute("SELECT id FROM users WHERE username = %s", (username,))
        if cursor.fetchone():
            print("Username already exists. Try another.")
        else:
            break
    while True:
        password = input("Choose a password: ")
        confirm = input("Confirm password: ")
        if password != confirm:
            print("Passwords do not match. Try again.")
        elif len(password) < 6:
            print("Password must be at least 6 characters.")
        else:

```

```

        break
    cursor.execute(
        "INSERT INTO users (username, password_hash) VALUES (%s, %s)", (username,
password)
    )
    cnx.commit()
    print("Registration successful! Please login now.")
    cursor.close()
    cnx.close()

def login_user():
    cnx, cursor = db_connect()
    print_banner("USER LOGIN")
    username = input("Enter username: ").strip()
    password = input("Enter password: ")
    cursor.execute("SELECT id, password_hash FROM users WHERE username = %s",
(username,))
    row = cursor.fetchone()
    if row is None:
        print("User not found.")
        cursor.close()
        cnx.close()
        return None
    user_id, stored_password = row
    if password == stored_password:
        print(f"Welcome back, {username}!")
        cursor.close()
        cnx.close()
        return user_id
    else:
        print("Incorrect password.")
        cursor.close()
        cnx.close()
        return None

def get_user_financial_summary(user_id):
    cnx, cursor = db_connect()
    cursor.execute(
        "SELECT SUM(amount) FROM entries WHERE user_id = %s AND type = 'income'",
(user_id,)
    )
    total_income = cursor.fetchone()[0] or 0
    cursor.execute(
        "SELECT SUM(amount) FROM entries WHERE user_id = %s AND type = 'expense'",
(user_id,)
    )
    total_expense = cursor.fetchone()[0] or 0
    cnx.close()
    return total_income, total_expense, total_income - total_expense

def record_entry(user_id):
    cnx, cursor = db_connect()

```



```

print_banner("RECORD INCOME/EXPENSE ENTRY")
while True:
    entry_type = input("Enter 'i' for income or 'e' for expense: ").lower()
    if entry_type == "i":
        entry_type_full = "income"
        break
    elif entry_type == "e":
        entry_type_full = "expense"
        break
    else:
        print("Please enter 'i' or 'e'.")
while True:
    try:
        amount = float(input("Enter amount (₹): "))
        if amount <= 0:
            print("Please enter a positive number.")
            continue
        break
    except:
        print("Invalid amount. Try again.")
if entry_type_full == "income":
    allowed_income_tags = [
        "salary",
        "bonus",
        "interest",
        "investment",
        "gift",
        "other",
    ]
    print(f"Allowed income tags: {' '.join(allowed_income_tags)}")
    while True:
        tag = input("Enter a tag for income: ").strip().lower()
        if tag in allowed_income_tags:
            break
        else:
            print("Invalid tag. Try again.")
else:
    tag = input("Enter a tag for expense: ").strip()
now = datetime.datetime.now()
cursor.execute(
    "INSERT INTO entries (user_id, type, amount, tag, entry_time) VALUES (%s, %s, %s, %s, %s)",
    (user_id, entry_type_full, amount, tag, now),
)
cnx.commit()
total_income, total_expense, balance = get_user_financial_summary(user_id)
if entry_type_full == "income":
    print("\n" + "=" * 60)
    print(f"🎉 Income recorded: ₹{amount:.2f} (Cr)")
    print(f"Total Income: ₹{total_income:.2f} (Cr)")
    print("=" * 60 + "\n")
    print("Hello! Your income entry is safely logged.")
else:
    print_banner("EXPENSE SUMMARY")
    print(f"Expense recorded: ₹{amount:.2f} (Dr), Tag: {tag}")

```

```

        print(f"Total Expense: ₹{total_expense:.2f} (Dr)")
        print(f"Current Balance: ₹{balance:.2f}")
        print("Well done staying on top of your expenses!")
    cursor.close()
    cnx.close()

def monthly_budget_planner(user_id):
    cnx, cursor = db_connect()
    print_banner("MONTHLY BUDGET PLANNER")
    try:
        budget = float(input("Enter your total monthly budget (₹): "))
        priorities = [input(f"Enter priority #{i+1} expense name: ") for i in
range(3)]
        percents = []
        for p in priorities:
            while True:
                try:
                    val = int(input(f"Enter % allocation for {p}: "))
                    if 0 <= val <= 100:
                        percents.append(val)
                        break
                    else:
                        print("Must be between 0 and 100")
                except:
                    print("Enter a valid integer")
            total_pct = sum(percents)
            if total_pct > 100:
                print("Percentages exceed 100%. Cannot proceed.")
                cursor.close()
                cnx.close()
                return
            misc_pct = 100 - total_pct
            current_month = datetime.datetime.now().strftime("%Y-%m")
            cursor.execute(
                "INSERT INTO monthly_budget (user_id, month, budget, priority1,
priority1_percent, priority2, priority2_percent, priority3, priority3_percent,
miscellaneous_percent) VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)",
                (
                    user_id,
                    current_month,
                    budget,
                    priorities[0],
                    percents[0],
                    priorities[1],
                    percents[1],
                    priorities[2],
                    percents[2],
                    misc_pct,
                ),
            )
        cnx.commit()
        print("\nBudget Breakdown:")
        print(f"{'Category'.ljust(25)}{'Amount'.rjust(15)}{'Percent'.rjust(10)}")
        print("-" * 50)

```

```

        for p, t in zip(priorities, percents):
            amt = budget * t / 100
            print(f"{p.ljust(25)}{str(round(amt, 2)).rjust(15)}{str(t).rjust(10)}%")
        misc_amt = budget * misc_pct / 100
        print(
            f"'Miscellaneous'.ljust(25)}{str(round(misc_amt, 2)).rjust(15)}{str(misc_pct).rjust(10)}%"
        )
        print("=" * 50)
        print("Great job on planning your budget for the month!")
    except Exception as e:
        print(f"Error: {e}")
    cursor.close()
    cnx.close()

```

```

def bill_splitter(user_id):
    cnx, cursor = db_connect()
    print_banner("BILL SPLITTER")
    try:
        total = float(input("Enter total bill amount: "))
        while True:
            people = input("Enter number of people: ")
            if people.isdigit() and int(people) > 0:
                people = int(people)
                break
            else:
                print("Enter a valid positive integer")
        share = total / people
        cursor.execute(
            "INSERT INTO bill_splits (user_D, bill_amount, num_people, per_person_share) VALUES (%s,%s,%s,%s)",
            (user_id, total, people, share),
        )
        cnx.commit()
        print(f"Each person should pay: ₹{share:.2f}")
        print("Bill split completed successfully!")
    except Exception as e:
        print(f"Error: {e}")
    cursor.close()
    cnx.close()

```

```

def tax_calculator(user_id):
    cnx, cursor = db_connect()
    print_banner("TAX CALCULATOR")
    try:
        income = float(input("Enter annual income (₹): "))
        slabs = [(250000, 0), (250000, 0.05), (500000, 0.20), (float("inf"), 0.30)]
        remaining = income
        tax = 0
        for slab, rate in slabs:
            taxable = min(remaining, slab)
            tax += taxable * rate
            remaining -= taxable
    except Exception as e:
        print(f"Error: {e}")
    cursor.close()
    cnx.close()

```

```

        if remaining <= 0:
            break
        effective_rate = (tax / income) * 100 if income > 0 else 0
        cursor.execute(
            "INSERT INTO tax_calculations (user_id, income, tax, effective_rate)
VALUES (%s,%s,%s,%s)",
            (user_id, income, tax, effective_rate),
        )
        cnx.commit()
        print(f"Tax due: ₹{tax:.2f}")
        print(f"Effective tax rate: {effective_rate:.2f}%")
        print("Tax calculation done. Keep your records updated!")
    except Exception as e:
        print(f"Error: {e}")
    cursor.close()
    cnx.close()

def compound_interest(user_id):
    cnx, cursor = db_connect()
    print_banner("COMPOUND INTEREST CALCULATOR")
    try:
        principal = float(input("Principal amount (₹): "))
        monthly_contrib = float(input("Monthly contribution (₹): "))
        years = float(input("Investment duration (years): "))
        rate = float(input("Annual interest rate (%): "))
        variance = float(input("Variance (%): "))
        freq_map = {"annual": 1, "quarterly": 4, "monthly": 12}
        freq = input("Compounding frequency (annual/quarterly/monthly): ").lower()
        if freq not in freq_map:
            print("Invalid frequency")
            cursor.close()
            cnx.close()
            return
        n = freq_map[freq]
        months = int(years * 12)
        rates = [rate - variance, rate, rate + variance]
        plt.figure(figsize=(10, 6))
        summaries = []
        for r in rates:
            balance = []
            amount = principal
            for month in range(months + 1):
                if n == 12:
                    monthly_rate = r / 100 / 12
                    amount = amount * (1 + monthly_rate) + monthly_contrib
                else:
                    if month % (12 // n) == 0 and month != 0:
                        monthly_rate = r / 100 / n
                        amount = amount * (1 + monthly_rate)
                    amount += monthly_contrib
                balance.append(amount)
            plt.plot(range(months + 1), balance, label=f"{r:.1f}%")
            interest_earned = balance[-1] - principal - monthly_contrib * months
            summaries.append(

```

```

        f"At {r:.1f}% total: {balance[-1]:.2f} (Interest earned:
{interest_earned:.2f})"
    )
    plt.title("Compound Interest Over Time")
    plt.xlabel("Months")
    plt.ylabel("Balance (₹)")
    plt.legend()
    plt.show()
    cursor.execute(
        "INSERT INTO compound_interest_records (user_id, principal,
monthly_contrib, years, annual_rate, variance, frequency, final_amount) VALUES
(%s,%s,%s,%s,%s,%s,%s,%s)",
        (user_id, principal, monthly_contrib, years, rate, variance, freq,
balance[-1]),
    )
    cnx.commit()
    print("\nSummary:")
    for s in summaries:
        print(s)
    print("Compound interest calculated! Your financial future looks bright!")
except Exception as e:
    print(f"Error: {e}")
cursor.close()
cnx.close()

```

```

def savings_calculator(user_id):
    cnx, cursor = db_connect()
    print_banner("SAVINGS CALCULATOR")
    try:
        saving = float(input("Monthly saving (₹): "))
        years = float(input("Duration (years): "))
        rate = float(input("Expected annual return (%): "))
        months = int(years * 12)
        r = rate / 100 / 12
        balances = [0]
        for _ in range(months):
            balances.append(balances[-1] * (1 + r) + saving)
        plt.figure(figsize=(10, 6))
        plt.plot(range(months + 1), balances, label="Savings Growth")
        plt.title("Savings Over Time")
        plt.xlabel("Months")
        plt.ylabel("Balance (₹)")
        plt.legend()
        plt.show()
        cursor.execute(
            "INSERT INTO savings_records (user_id, monthly_saving, years,
annual_return, final_balance) VALUES (%s,%s,%s,%s,%s)",
            (user_id, saving, years, rate, balances[-1]),
        )
        cnx.commit()
        print(f"Total savings after {years} years: ₹{balances[-1]:.2f}")
        print("Savings goal on track! Keep it up!")
    except Exception as e:
        print(f"Error: {e}")

```

```

cursor.close()
cnx.close()

def currency_converter(user_id):
    cnx, cursor = db_connect()
    print_banner("CURRENCY CONVERTER")
    print("Supported currencies:")
    for k, v in CURRENCIES.items():
        print(f"{k} - {v}")

    c = CurrencyRates()
    try:
        while True:
            amount = float(input("Enter amount: "))
            from_curr = input("From currency code: ").upper()
            to_curr = input("To currency code: ").upper()
            if from_curr not in CURRENCIES or to_curr not in CURRENCIES:
                print("Invalid currency code.")
                continue

            use_fixed_rates = input(
                "Use fixed offline rates instead of live rates? (yes/no): "
            ).lower()

            if use_fixed_rates == "yes":
                try:
                    # Conversion logic using fixed rates
                    if from_curr == "INR":
                        converted = convert_inr_to_currency(amount, to_curr)
                    elif to_curr == "INR":
                        converted = convert_currency_to_inr(amount, from_curr)
                    else:
                        # Convert from from_curr to INR, then INR to to_curr
                        inr_amount = convert_currency_to_inr(amount, from_curr)
                        converted = convert_inr_to_currency(inr_amount, to_curr)
                    print(f"{amount:.2f} {from_curr} = {converted:.4f} {to_curr}")
                except ValueError as ve:
                    print(ve)
                    continue
            else:
                # Live conversion using API
                converted = c.convert(from_curr, to_curr, amount)
                print(f"{amount:.2f} {from_curr} = {converted:.2f} {to_curr}")

            cursor.execute(
                "INSERT INTO currency_conversions (user_id, from_currency, "
to_currency, amount, converted_amount) VALUES (%s,%s,%s,%s,%s)",
                (user_id, from_curr, to_curr, amount, converted),
            )
            cnx.commit()
            print("Currency conversion successful!")
            again = input("Convert another? (yes/no): ").lower()
            if again != "yes":
                break

```

```

except Exception as e:
    print(f"Error: {e}")
cursor.close()
cnx.close()

def main_menu():
    print_banner("USER MENU")
    print("1. Monthly Budget Planner")
    print("2. Bill Splitter")
    print("3. Tax Calculator")
    print("4. Compound Interest Calculator")
    print("5. Currency Converter")
    print("6. Savings Calculator")
    print("7. Record Income/Expense")
    print("8. Logout")
    return input("Select an option (1-8): ")

def main():
    create_all_tables()
    print_banner("WELCOME TO COLLEGE FINANCE BOT")
    user_id = None
    while True:
        choice = input("Are you a new user or returning? (new/login): ").lower()
        if choice == "new":
            register_user()
        elif choice == "login":
            user_id = login_user()
            if user_id:
                print("Hello! Ready to assist you with your finances.")
                break
        else:
            print("Invalid choice.")
    while True:
        choice = main_menu()
        if choice == "1":
            monthly_budget_planner(user_id)
        elif choice == "2":
            bill_splitter(user_id)
        elif choice == "3":
            tax_calculator(user_id)
        elif choice == "4":
            compound_interest(user_id)
        elif choice == "5":
            currency_converter(user_id)
        elif choice == "6":
            savings_calculator(user_id)
        elif choice == "7":
            record_entry(user_id)
        elif choice == "8":
            print("Goodbye! Have a great day!")
            break
        else:
            print("Invalid option.")

```

```
if __name__ == "__main__":  
    main()
```